LESSON 03 · CHEAT SHEET

Functions

DEFINE, THEN CALL

```
def monthly_salary(annual):   # def name(params):
    """One twelfth, to the penny."""  # docstring
    return round(annual / 12, 2)  # hand back

m = monthly_salary(32500)        # call: brackets!
print(m)                         2708.33
```

VBA: `Function MonthlySalary(annual) …` `MonthlySalary = … End Function`. Define **above** the first call.

RETURN VS PRINT

```
def money(x):
    return f"{x:,.2f} cr"    # caller gets the text

def show(x):
    print(f"{x:,.2f} cr")    # caller gets None

money(5) + " net"            '5.00 cr net'
show(5) + " net"             # TypeError: None + str
```

No `return` (or a bare `return`) means the function returns `None`. Return values; print at the end.

EARLY RETURN

```
def bonus_rate(months, rating):
    if months < 12:
        return 0          # function ends here
    if rating >= 4:
        return 0.08
    if rating == 3:
        return 0.04
    return 0              # everything else
```

DEFAULTS & KEYWORDS

```
def percent(x, decimals=1):
    return f"{x * 100:.{decimals}f}%"

percent(0.125)               '12.5%'
percent(0.125, 0)            '12%'
percent(0.125, decimals=2)   '12.50%'

def pay_scroll(name, salary, pension=0.05, tax=0.2):
    ...
pay_scroll("Orrin", 36500, pension=0.08) # keyword
```

Required params first, defaults after. Keywords can come in any order.

FUNCTIONS CALLING FUNCTIONS

```
def money(x):
    return f"{x:,.2f} cr"

def scroll_line(label, amount):
    return f"{label:<15}{money(amount):>12}"

def bonus(salary, months, rating):
    return salary * bonus_rate(months, rating)
```

One rule, one place. Change `money` and every line follows.

SCOPE: INSIDE STAYS INSIDE

```
rate = 0.05                  # outside: seen by all

def pension(basic):
    deduction = basic * rate # can READ rate
    return deduction         # local: only in here

def other(basic, rate):      # its own rate wins
    return basic * rate

print(deduction)             # NameError
```

Need something from inside? `return` it. Need something inside? Pass it as a parameter.

TRY FUNCTIONS IN THE REPL

```
python -i my_work\ex5_holiday.py
>>> holiday_days(6)
28
>>> holiday_days(6, base=28)
31
>>> help(holiday_days)       # shows the docstring
```

ONE-LINE IF

```
unit = "year" if n == 1 else "years"
```

GOTCHAS

- No `return` → you get `None`
- `money` vs `money(50)`: recipe vs result
- Call before `def` → NameError
- Code after `return` never runs
- Body not indented → IndentationError
- Changing a parameter inside doesn't change the outside variable